

Pointer

*name of
variable*

*storage
address*

content

a



0000

0001

0002

0003

0004

...

...

1004

1005

b

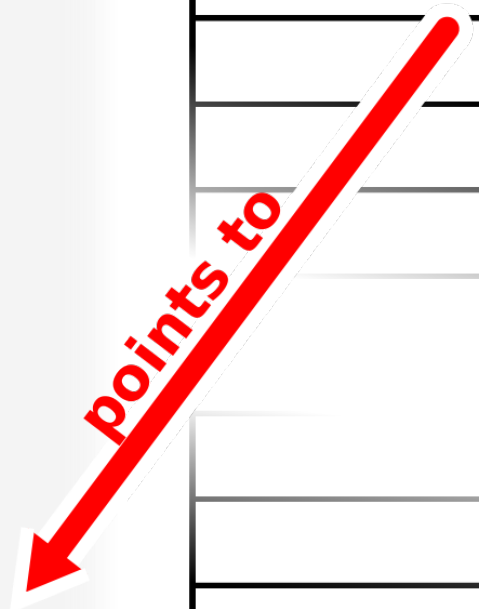


1008

1009

1010

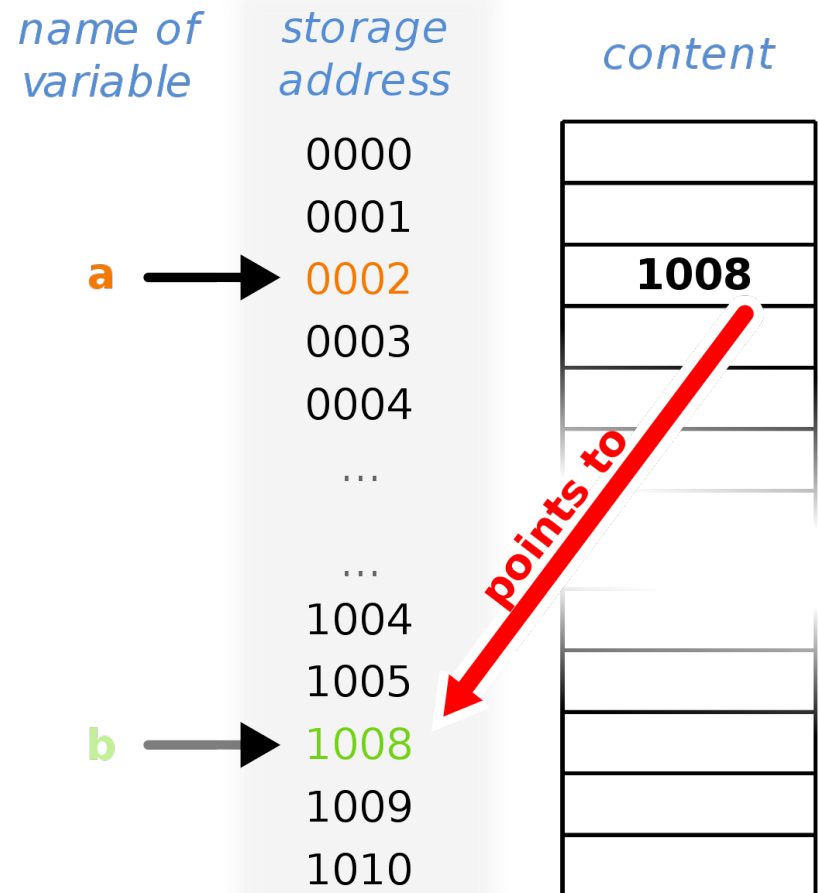
1008



What is Pointer?

❖ A **pointer** is a variable whose value is the address of another variable.

- Computer memory is divided into **byte** for addressing.
- Each byte is numbered sequentially- which is called **storage address**.
- Storage address is represented in **hexadecimal** number system. In hexadecimal, following symbols are used: **A for 10**, B for 11, **C for 12**, D for 13, **E for 14** and F for 15.



What is Pointer?

Declaration:

```
int *a, b;
```

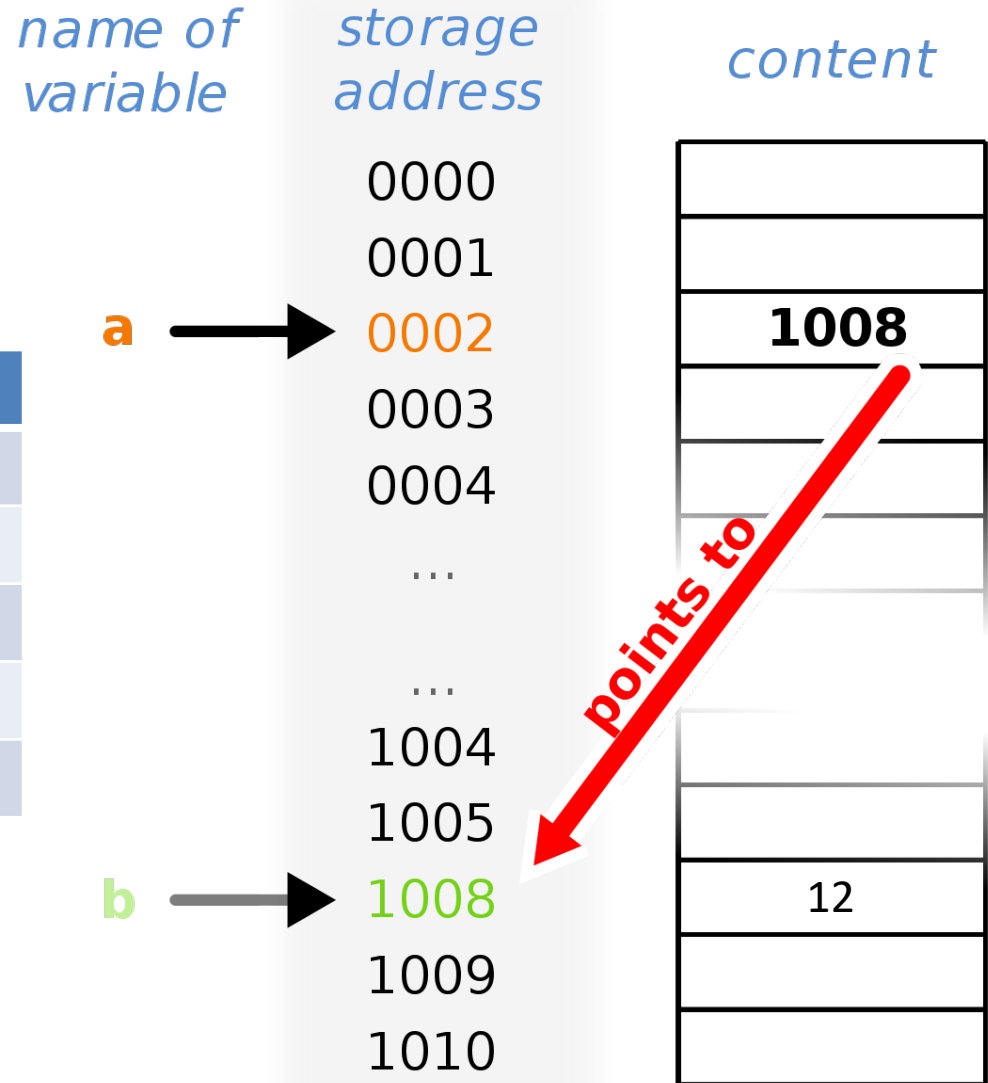
Understanding point:

Question	Answer
&a	0002
a	1008
*a	12
&b	1008
b	12

Read out:

*a: pointer of a

&a: ampersand or address of a



Pointer Examples 1

Write down the output of the following program:

```
#include <stdio.h>
```

```
main(){
```

```
    int u = 3, v, *pu, *pv;
```

```
    pu = &u;
```

```
    v = *pu;
```

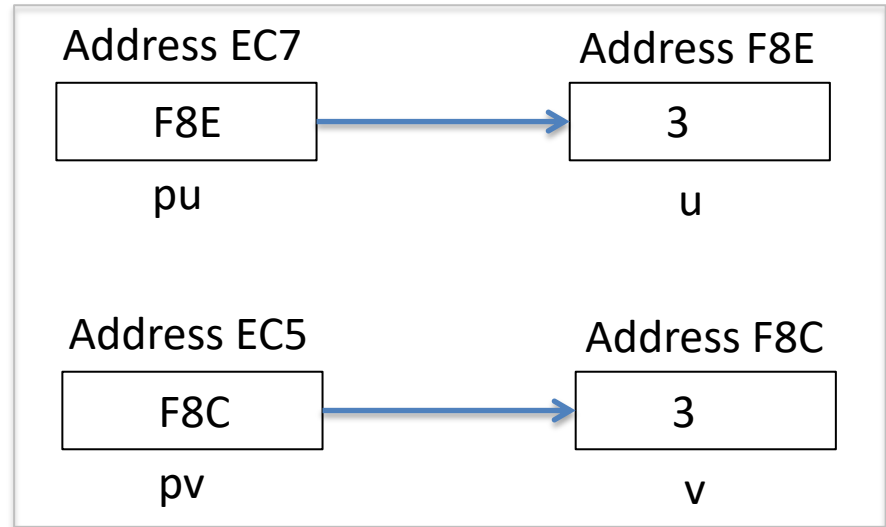
```
    pv = &v;
```

```
    printf("\nu=%d    &u=%x    pu=%x    *pu=%d", u, &u, pu, *pu);
```

```
    printf("\nv=%d    &v=%x    pv=%x    *pv=%d", v, &v, pv, *pv);
```

```
}
```

Assume that, &pu = EC7, &u = F8E, &pv = EC5, &v = F8C



Output

u = 3	&u = F8E	pu = F8E	*pu = 3
v = 3	&v = F8C	pv = F8C	*pv = 3

Pointer Examples 2

Write down the output of the following program:

```
#include <stdio.h>
```

```
main(){
```

```
    int u1, u2;
```

```
    int v = 3, *pv;
```

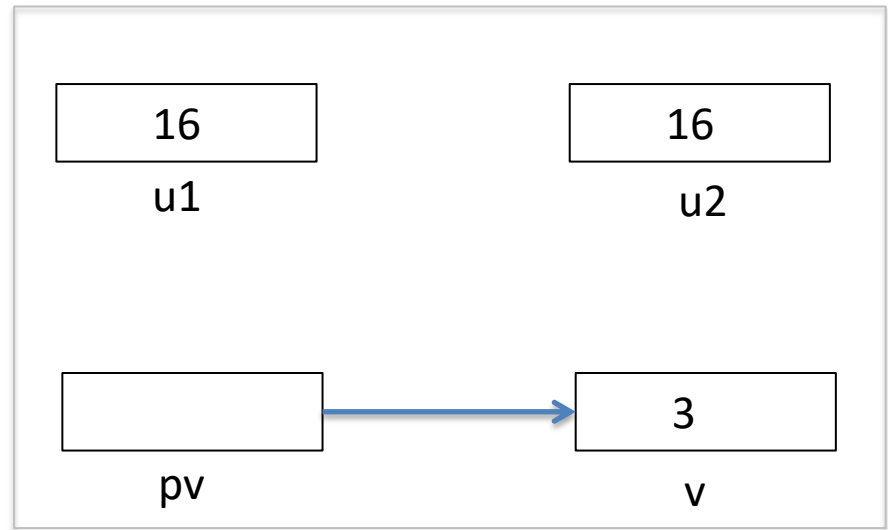
```
    u1 = 2 * ( v + 5 );
```

```
    pv = &v;
```

```
    u2 = 2 * ( *pv + 5 );
```

```
    printf("\nu1=%d    u2=%d", u1, u2);
```

```
}
```



Output

u1 = 16 u2 = 16

Pointer Examples 3

Write down the output of the following program:

```
#include <stdio.h>
```

```
main(){
```

```
    int v = 3;
```

```
    int *pv;
```

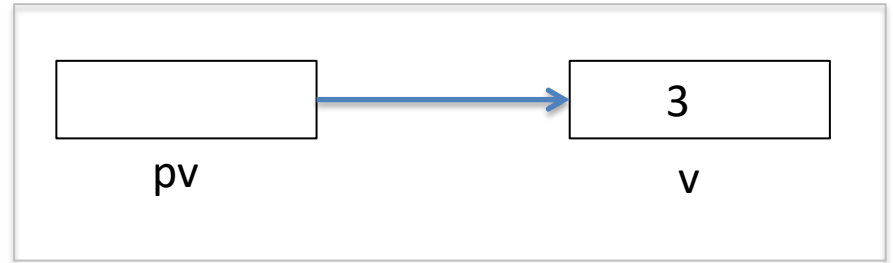
```
    pv = &v;
```

```
    printf("\n*pv=%d    v=%d", *pv, v);
```

```
    *pv = 0;
```

```
    printf("\n*pv=%d    v=%d", *pv, v);
```

```
}
```

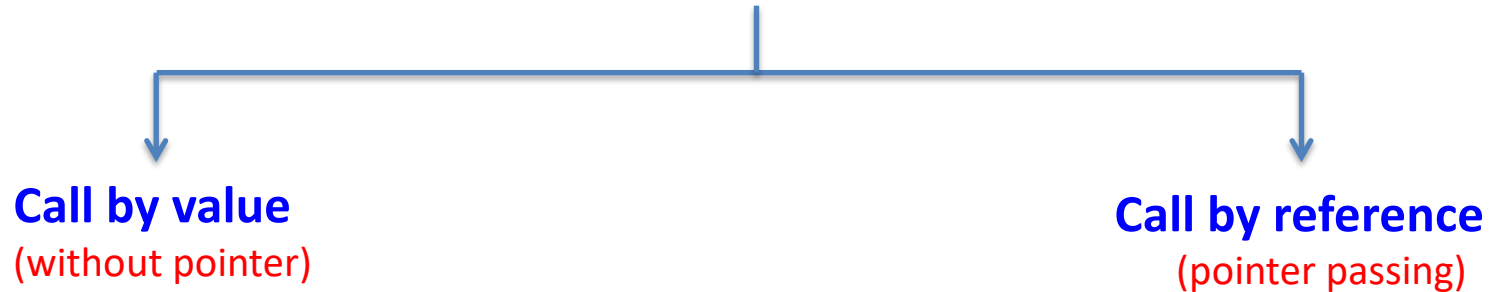


Output

```
*pv = 3    v = 3
*pv = 0    v = 0
```

Passing Pointer to Function

Function Arguments/Parameters



- ❖ **Call by value**: passing the value of a variable to a function as an argument.

```
int funct(int u, int v);
```

- ❖ **Call by reference**: passing the address of a variable to a function as an argument.

```
int funct(int *pu, int *pv);
```

Call by value & call by reference Example

Write down the output of the following program:

```
#include <stdio.h>
```

```
void funct1(int u, int v);
```

```
void funct2(int *pu, int *pv);
```

```
main(){
```

```
    int u = 1, v = 3;
```

```
    printf("\nBefore calling funct1:  u = %d   v = %d", u, v);
```

```
    funct1(u, v);
```

```
    printf("\n\nAfter calling funct1:  u = %d   v = %d", u, v);
```

```
    printf("\nBefore calling funct2:  u = %d   v = %d", u, v);
```

```
    funct2(&u, &v);
```

```
    printf("\nAfter calling funct2:  u = %d   v = %d", u, v);
```

```
}
```

Address EC7	Address F8E
1	3
u	v

Call by value & call by reference Example Continue..

```
void funct1(int u, int v){
```

```
    u = 0;
```

```
    v = 0;
```

```
    printf("\nWithin funct1:    u = %d    v = %d", u, v);
```

```
}
```



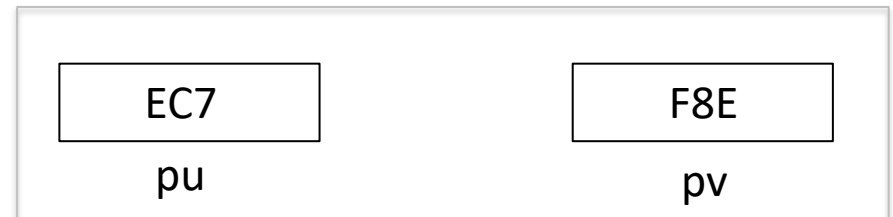
```
void funct2(int *pu, int *pv){
```

```
    *pu = 0;
```

```
    *pv = 0;
```

```
    printf("\nWithin funct2:    *pu = %d    *pv = %d", *pu, *pv);
```

```
}
```



Call by value & call by reference Example Continue..

Output:

Before calling funct1: u = 1 v = 3

Within funct1: u = 0 v = 0

After calling funct1: u = 1 v = 3

Before calling funct2: u = 1 v = 3

Within funct2: *pu = 0 *pv = 0

After calling funct2: u = 0 v = 0

Pointer and Array

- ❖ An **array name** is a variable which points to some consecutive memory location.

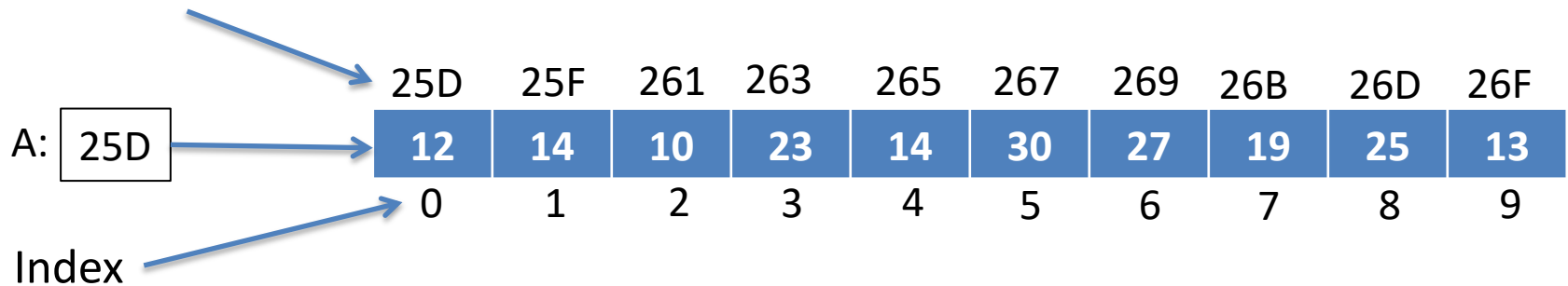
Declaration of 1D Array:

```
int A[10];
```

Required memory size:

char	1 byte
int	2 bytes
float	4 bytes

Memory address



Example:

$A[0] = 12$

$A[3] = 23$

$A[7] = 19$

$*A = 12$

$*(A + 3) = 23$

$*(A + 7) = 19$

$A = 25D$

$(A + 3) = 263$

$(A + 7) = 26B$

Pointer and Array

Example:

For **char** pointer, `char *A;`



$A + 1 = A + \text{size of char}$

$A + 5 = A + 5 * \text{size of char}$

For **int** pointer, `int *A;`



$A + 1 = A + \text{size of int}$

$A + 3 = A + 3 * \text{size of int}$

Memory size:

char	1 byte
int	2 bytes
float	4 bytes

For **float** pointer, `float *A;`



$A + 1 = A + \text{size of float}$

$A + 2 = A + 2 * \text{size of float}$

Pointer Operation

```
int *pa, *pb;
```

Operation	Whether permitted or not ?
$pa + 5$	Permitted, move to 5 elements ahead
$pa - 3$	Permitted, move to 3 elements behind
$pa - pb$	Permitted, finds the number of elements between the address
$pa + pb$	Not permitted
$pa * pb$	Not permitted
pa / pb	Not permitted

Pointer and Array

Declaration of 1D Array:

```
int A[10];
```



Declaration of 1D Array using Pointer:

step 1. Declaration of pointer

```
int *A;
```



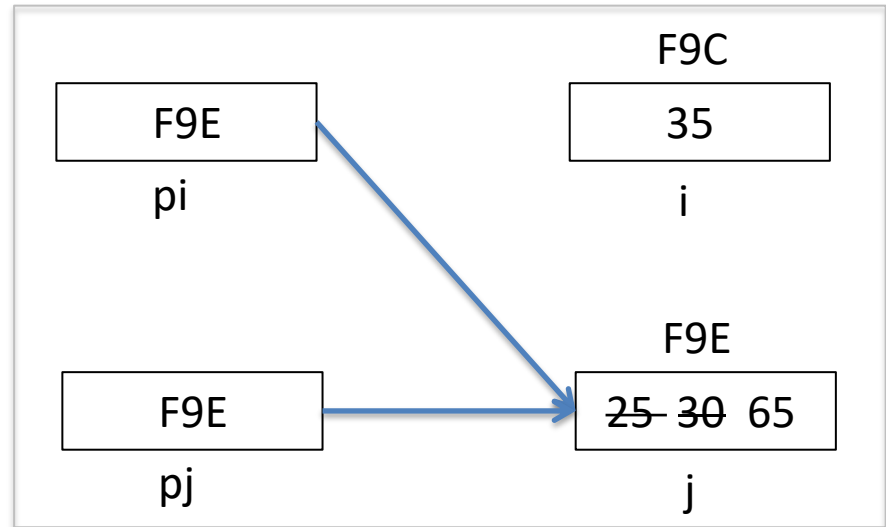
step 2: Allocation of Memory

```
A = (int *) malloc(10*sizeof(int));
```



Pointer Exercise 1

```
int i, j = 25;  
int *pi, *pj = &j;  
.....  
*pj = j + 5;  
i = *pj + 5;  
Pi = pj;  
*pi = i + j;
```



The value assigned to `i` begins at address `F9C` and the value assigned to `j` begins at address `F9E`.

Output

`&i = F9C`

`&j = F9E`

`pj = F9E`

`*pj = 65`

`i = 35`

`pi = F9E`

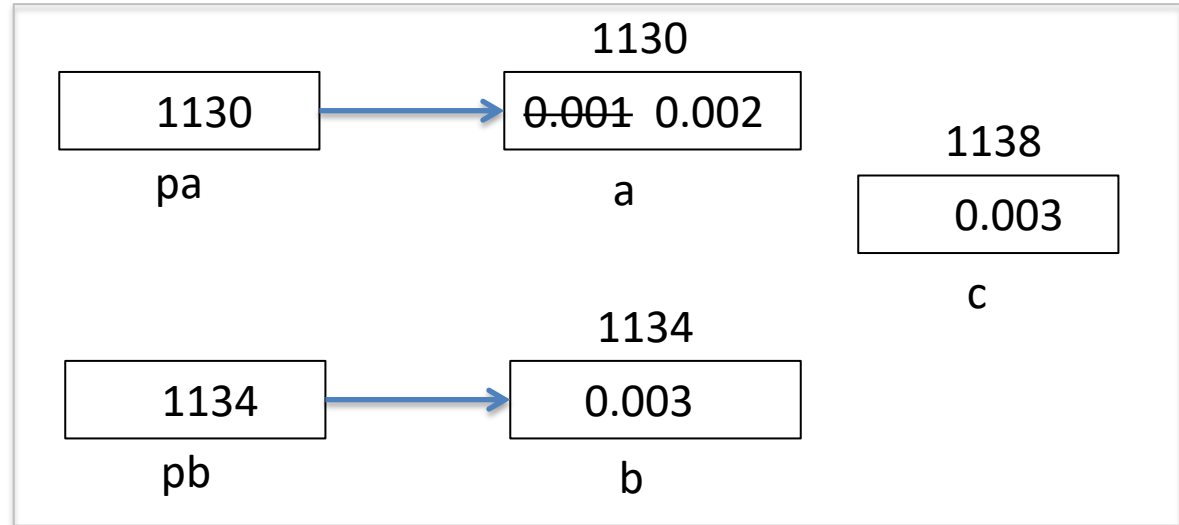
`*pi = 65`

`(pi + 2) = F9E + 4 = FA2`

`*(pi + 2) = unknown.`

Pointer Exercise 2

```
float a = 0.001, b = 0.003;  
float c, *pa, *pb;  
pa = &a;  
*pa = 2 * a;  
pb = &b;  
c = 3 * (*pb - *pa);
```



The value assigned to `a` begins at address 1130 and the value assigned to `b` begins at address 1134 and the value assigned to `c` begins at 1138.

Output

&a = 1130

&b = 1134

&c = 1138

pa = 1130

*pa = 0.002

&>(*pa) = 1130

pb = 1134

*pb = 0.003

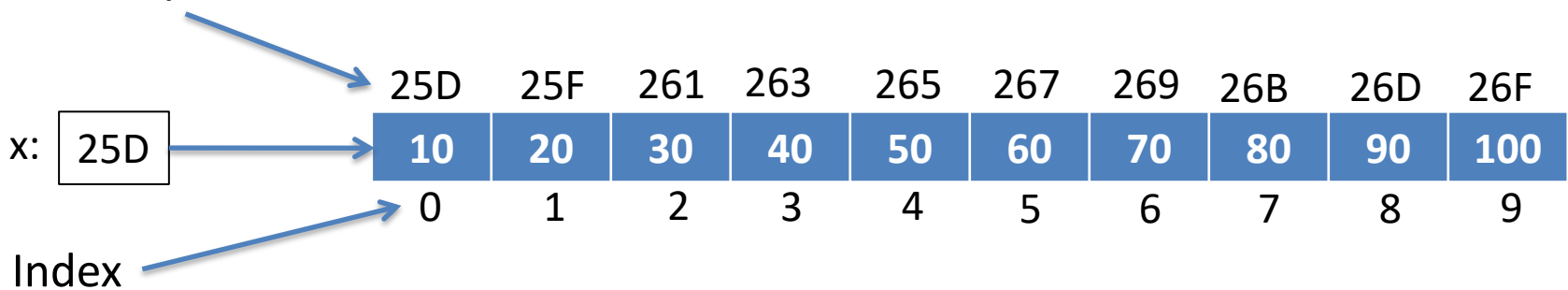
c = 0.003

Pointer Exercise 3

```
static int x[10] = { 10, 20, 30, 40, 50, 60, 70, 80, 90, 100 }
```

The value assigned to x begins at address 25D.

Memory address



Output

$x = 25D$

$(x + 2) = 25D + 2 * 2 = 261$

$*x = 10$

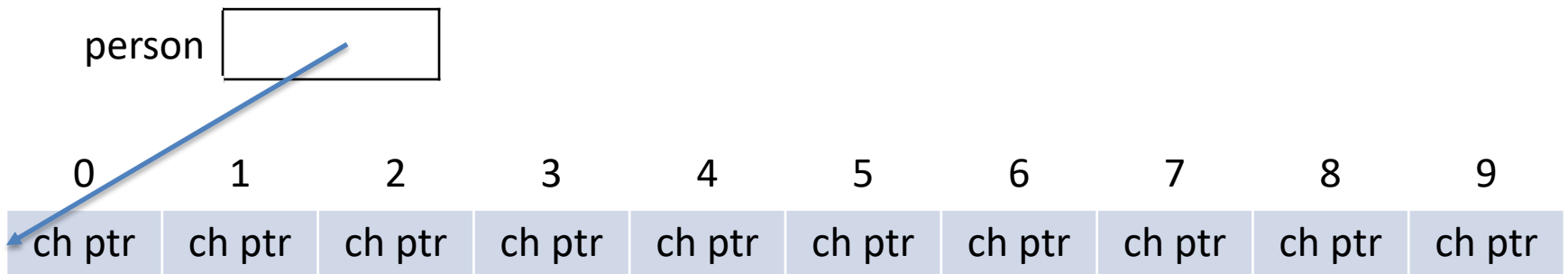
$(*x + 2) = 10 + 2 = 12$

$*(x + 2) = 30$

Pointer and 2D Array

❖ *** operator** has a lower precedence than the **[] operator**.

Declaration 1: `char *person[10];` //Array of pointers



Declaration 2: `int (*A)[5];` //Pointer to an array



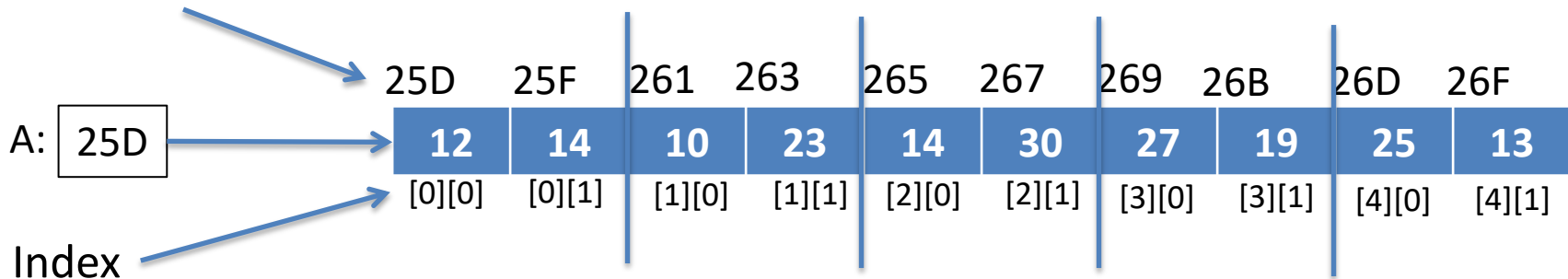
- This is useful for processing two-dimensional array parameters declared with **unknown number of rows**.

Pointer and 2D Array

Declaration of 1D Array:

```
int A[5][2];
```

Memory address



Example:

$A[2][1] = 30$

$*(*(A+2) + 1) = 30$

$A = 25D,$

$*A = 25D,$

$**A = 12$

$(A+1) = 261,$

$*A + 1 = 25F,$

$*(A+1) = 261$

Pointer and 2D Array

Declaration of 2D Array using Pointer:

step 1. Declaration of pointer

```
int **A;
```

A: 

step 2: Allocation of row

```
A = (int **) malloc( row * col * sizeof(int) );
```

```
int A[M][N];
```

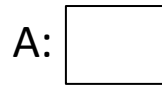
Address of A[row][col] = A + (N*row + col) * sizeof(int);

Pointer and 2D Array

Declaration of 2D Array using Pointer (alternative way):

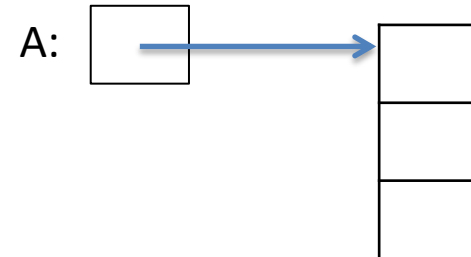
step 1. Declaration of pointer

```
int **A;
```



step 2: Allocation of row

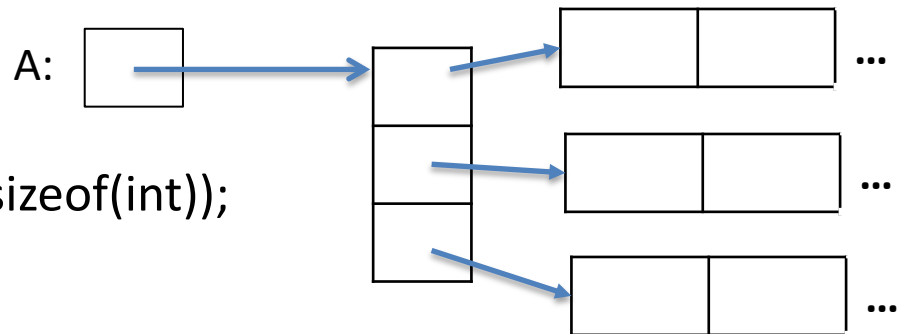
```
A = (int **) malloc(row*sizeof(int *));
```



step 3: Allocation of Memory

```
for ( i = 0; i < row; ++i)
```

```
    A[i] = (int *) malloc(col*sizeof(int));
```



Pointer and function

`int *func_name ( int a );` `//function returns a pointer to an integer`

`int (*func_name)( int a );` `// pointer to a function`

Try Yourself.....

- (1) `int *(*f)( int a );`
- (2) `int *(*p)(int (*a)[]);`
- (3) `int *(*p)(int *a[]);`
- (4) `char *f(char *a);`